# Lecture 6: Recurrent Neural Networks, Decoding

Andrei Arsene Simion

Columbia University

October 7, 2025

# Outline

- Language Modeling
- Recurrent Neural Networks
- LSTM
- Decoding

# Outline

- Language Modeling
- Recurrent Neural Networks
- LSTM
- Decoding

# Introduction to Language Models

- Language Modeling is one of the oldest problems in NLP
- Nowadays, language models are usually termed "Foundation" Models since people are starting to realize that solving Language Modeling leads to solving other things
- At a high level, we'd like to be able to construct plausible sentences, sensible sequences $(y_1, \ldots, y_T)$ with $p(y_1, y_2, \ldots, y_T)$ high
- This can be done with n-gram count based models (we will see one soon), CNNs (touched on this before), and even a standard MLP (we will see this also)
- Finally, we will see how this is done with RNNs and we will find out how to "generate" from an Language Model a sentence

# Language Models: Why?

- Language models in some sense allow us to understand how words follow each other and have many applications
  - Given a question, tell me the answer (identify the answer in some context)
  - Given the start of a sentence, tell me the next few words (generate text)
  - From a language model, we might extract *contextual* word representations (vectors) and apply them to other things - we'll see this later
  - Etc.

# The vocabulary set $\mathcal{V}$

- First, we will define $\mathcal{V}$ to be the set of all words in the language
- For example, this can be $\mathcal{V}$={the, dog, laughs, saw, barks, cat, ...}
  - Assume $\mathcal{V}$ can be quite large
  - Assume $\mathcal{V}$ is finite
- Given this corpus, we'd like to estimate the parameters of a Language Model
- A *sentence* is a sequence of words

$$x_1 x_2 \ldots x_{T-1}$$

where $T \geq 1$ and $x_t \in \mathcal{V}$ for $t \in \{1, \ldots, T-1\}$

# Language Models

- We assume that $x_T$ is a special symbol STOP which is not in $\mathcal{V}$
  - Depending on the model, we also have a symbol START which we will use; this is also not in $\mathcal{V}$
  - This will typically be for $x_0$ or $x_{-1}$ as we will see
  - So: all sentences begin with START and end in STOP
  - A sentence $(x_1 x_2 \ldots x_{T-1})$ can then have the padded form $(x_0, x_1, \ldots, x_T)$ or $(x_{-2}, x_{-1}, x_0, x_1, \ldots, x_T)$ Etc.
- Some example sentences are as follows

START the dog barks STOP

START the STOP

START STOP

START START cat cat cat STOP

START the cat saw the dog STOP

- Notice: Some sentences make more sense than others! Good sentences should have a higher "score"

# Language Models: The probability is the score

- Given a vocabulary $\mathcal{V}$ there is an infinite set of sentences $\mathcal{S}$ of the form $x_0, x_1 \ldots x_T$
- Each sentence has a probability $p(x_0 \ldots x_T)$ and a assume that $\sum_{x \in \mathcal{S}} p(x) = 1$
- The above defines a Language Model
- For example
  - Assume we have a bunch of sentences in a training corpus
  - For each sentence, define $p(x) = \frac{count(x)}{|\mathcal{S}|}$, where $count(x)$ is the number of times we saw the sentence $x$ in the corpus
- Obviously, many sentences will have a probability 0, and this is a bad model
- How can we do better? How can we say one model is better than another?

# Markov Language Models

- Markov Language Models are some of the oldest models whose goal is to do better than the above
- To start, let is look at $p(x_0, \ldots, x_T)$ again and apply Bayes' rule in its "chain rule" way
- Recall the chain rule:
  - $p(x, y) = p(y|x)p(x)$
  - $p(x, y, z) = p(z|y, x)p(x, y) = p(z|y, x)p(y|x)p(x)$
  - $p(x, y, z, w) = p(w|z, y, x)p(z|y, x)p(y|x)p(x)$
  - Etc.
- If we do this over and over, we get

$$p(x_0, x_1, \ldots, x_T) = (\prod_{t=1}^{T} p(x_t|x_{t-1}, \ldots, x_0))p(x_0)$$

# Markov Language Models

- We generally have

$$p(x_0, \ldots, x_T) \sim \prod_{t=1}^{T} p(x_t | x_{t-1}, \ldots, x_0)$$

- In Markov Models, we assume that $p(x_t | x_{t-1}, \ldots, x_0)$ depends on the past in a finite way
  - Bigram Model: $p(x_t | x_{t-1}, \ldots, x_1) = p(x_t | x_{t-1})$
  - Trigram Model: $p(x_t | x_{t-1}, \ldots, x_1) = p(x_t | x_{t-1}, x_{t-2})$
  - Fourgram Model: $p(x_t | x_{t-1}, \ldots, x_1) = p(x_t | x_{t-1}, x_{t-2}, x_{t-3})$
  - Etc
- This is the Markov Assumption: the future depends on the the immediate past, where "immediate" can mean 1/2/3 Etc time steps

# Final Note On Bayes' Rule

- Final note on

$$p(x_0, \ldots, x_T) \sim \prod_{t=1}^{T} p(x_t | x_{t-1}, \ldots, x_0)$$

- If $T = 2$ the above is $p(x_0, x_1, x_2)$ equals $p(x_2|x_1, x_0)p(x_1|x_0)$ and notice that $p(x_0)$ is missing
- However, for any sentence $x \in \mathcal{S}$ we have a term $p(x_0)$ and we can also fix $p(x_0)$; every sentence starts with this padding $x_0 = \text{START}$ (or more)
- So, we can drop this constant when comparing sentence probabilities
- We can just renormalize as needed ...

# Trigram Parameter Estimation

- Let's fix the Trigram Language Model so that

$$p(x_t|x_{t-1}, \ldots, x_0, x_{-1}) = p(x_t|x_{t-1}, x_{t-2})$$

  - Note that we have $x_0 = x_{-1}$=START since for any $x_t$ we have these two words coming before
  - Note that in this case, we need 2 padding tokens START for each sentence if we apply this to $x_1$
- Training? The goal is to estimate $p(x_t|x_{t-1}, x_{t-2})$ given some data ... We'll see this in a bit
- What about inference? How would we simulate from this model if we had parameters?

Step 1 Set $t = 1$ and $x_0 = x_{-1} = \text{START}$
Step 2 Generate $x_t$ from $p(x_t|x_{t-1}, x_{t-2})$
Step 3 If $x_t = \text{STOP}$, return $x_1, \ldots, x_{t-1}$
Step 4 Else, set $t = t + 1$ and go to Step 1

# Trigram Parameter Estimation

- Assume we have the Trigram language model, whose main parameters are $\theta(w|v, u)$
- Assume that $w \in \{\mathcal{V} \cup \{\text{STOP}\}\}$ and $u, v \in \{\mathcal{V} \cup \{\text{START}\}\}$
- The value $\theta(w|v, u)$ can be interpreted as seeing $w$ after seeing $(u, v)$
- For any sentence $x = (x_1, \ldots, x_T)$ we have that

$$p(x) = \theta(x_1|x_0, x_{-1})\theta(x_2|x_1, x_0) \ldots \theta(x_T|x_{T-1}, x_{T-2})$$

- Recall that in the above $x_0 = x_{-1} = \text{START}$ and $x_T = \text{STOP}$

# Trigram Language Model Example

- For example, suppose $x =$ the man STOP and we add two padding tokens $x_0 = x_{-1} =$START
  - The above has probability

$$\theta(\text{the}|\text{START}, \text{START})\theta(\text{man}|\text{the}, \text{START})\theta(\text{STOP}|\text{the}, \text{man})$$

- The parameters of this model satisfy $\theta(w|v, u) \geq 0$ for any $(u, v, w)$ and

$$\sum_{w \in \{\mathcal{V} \cup \{\text{STOP}\}\}} \theta(w|v, u) = 1$$

- The key is to figure out a way to get good estimates for $\theta(w|v, u)$
  - $w \in \{\mathcal{V} \cup \{\text{STOP}\}\}$
  - $u \in \{\mathcal{V} \cup \{\text{START}\}\}$ and $v \in \{\mathcal{V} \cup \{\text{START}\}\}$
  - Notice that $\theta(man|\text{START}, \text{the})$ does not make sense; $v$ can be START only if $u$ is
    - $|\mathcal{V}|^3$, the number of parameters, can be quite large; $|\mathcal{V}| = 10000$ is small, $O(|\mathcal{V}|^3)$ is massive

# Parameter Estimation and the Trigram Language Model

- How do we estimate the terms $\theta(w|v,u)$ from data?
- Assume we have sentences $\{x^{(i)}\}_{i=1}^{N}$ where each sentence can have a different length $T_i$
- Given this, we can form the Log-Likelihood (Loss) function

$$\mathcal{L}(\theta) = -\frac{\sum_{i=1}^{N} \log p(x^{(i)})}{N} = -\frac{\sum_{i=1}^{N} \log \prod_{t=1}^{T_i} \theta(x_t^{(i)}|x_{t-1}^{(i)}, x_{t-2}^{(i)})}{N}$$

- Using $\log(ab) = \log a + \log b$, this is

$$\mathcal{L}(\theta) = -\frac{\sum_{i=1}^{N} \sum_{t=1}^{T_i} \log \theta(x_t^{(i)}|x_{t-1}^{(i)}, x_{t-2}^{(i)})}{N}$$

- We'd like to minimize $\mathcal{L}(\theta)$ subject to constraints of the form $\theta(w|v,u) \geq 0$ for any $(u,v,w)$ and

$$\sum_{w \in \{\mathcal{V} \cup \{\text{STOP}\}\}} \theta(w|v,u) = 1$$

# Parameter Estimation and the Trigram Language Model

- Recall that

$$\mathcal{L}(\theta) = -\frac{\sum_{i=1}^{N} \sum_{t=1}^{T_i} \log \theta(x_t^{(i)} | x_{t-1}^{(i)}, x_{t-2}^{(i)})}{N}$$

- Before moving on, notice that

$$\mathcal{L}(\theta) = -\frac{\sum_{(u,v,w)} count(u, v, w) \log \theta(w|v, u)}{N}$$

- Above, $count(u, v, w)$ is the number of times we've seen the Trigram $(u, v, w)$ in the corpus
- We have some constraints:
  - In the above $u \in \{\mathcal{V} \cup \text{START}\}$
  - In the above $w \in \{\mathcal{V} \cup \text{STOP}\}$
  - If $u = \text{START}$, $v \in \{\mathcal{V} \cup \text{START}\}$
  - If $u \in \{\mathcal{V}\}$, $v \in \{\mathcal{V}\}$

# Trigram Language Model Lagrange Multipliers

- A typical way to do this is to introduce Lagrange multipliers $\lambda(u, v)$ and bring the constraints into the objective as

$$\mathcal{L}(\theta, \lambda) = -\frac{\sum_{(u,v,w)} count(u, v, w) \log \theta(w|v, u)}{N} + \sum_{(u,v)} \lambda(u, v)[\sum_{w} \theta(w|v, u) - 1]$$

- Consider the partial derivative of this with respect to $\theta(w|v, u)$ for some fixed $(u, v, w)$
- Let $\theta^\star$ and $\lambda^\star$ be the optimal parameters

# Parameter Estimation and the Trigram Language Model

- Thinking about this, we get that this partial derivative is

$$\frac{\partial \mathcal{L}}{\partial \theta(w|v,u)} = -\frac{count(u,v,w)}{N\theta^\star(w|v,u)} + \lambda^\star(u,v) = 0$$

- This gives $\theta^\star(w|v,u) = \frac{count(u,v,w)}{N\lambda^\star(u,v)}$
- Since we have $\sum_{w \in \{\mathcal{V} \cup \{STOP\}\}} \theta^\star(w|v,u) = 1$
- If we sum and use $\sum_w count(u,v,w) = count(u,v)$ we get

$$1 = \frac{count(u,v)}{N\lambda^\star(u,v)}$$

- This means that $\frac{count(u,v)}{N} = \lambda^\star(u,v)$
- Hence, we get ($N$ cancels) $\theta^\star(w|v,u) = \frac{count(u,v,w)}{count(u,v)}$

# Parameter Estimation and the Trigram Language Model

- So, the Maximum Likelihood estimates for these parameters are

$$\theta^{\star}(w|v,u) = \frac{count(u,v,w)}{count(u,v)}$$

  - $count(u,v,w)$ is the number of times we see this triplet of word tokens in the training corpus (in order)
  - $count(u,v)$ is the number of times we see this pair of word tokens in the training corpus (in order)
- Issues? These models are old-school and they had a few
  - If $count(u,v,w)$ is zero, does that mean $\theta(w|v,u) = 0$ ... Should this be the case?
  - If $count(u,v)$ is zero, how do we even estimate $\theta(w|v,u)$ ?
- If interested ask me but there were some ways around this with smoothing and back-up ...

# Perplexity: How to measure Language Model Goodness

- How do we know if a Language Model is good?
- Ideally, it should give a high probability to a good sentence, and a low probability to a bad sentence
- We always minimize the Loss $\mathcal{L} = -\frac{\sum_{i=1}^{N} \log p(x^{(i)})}{N}$, but in NLP we also use another metric called the Perplexity to quantify model goodness
- Let $T$ be the sum of all Test sentence lengths and $N$ be the number of Test sentences
- Then we have

$$\text{Perplexity} = 2^{\frac{N\mathcal{L}}{T}}$$

- What is the min / max of Perplexity?
  - The exponent is of the form $-\log \text{probability}$ so $\frac{N\mathcal{L}}{T}$ is between 0 and $\infty$
  - A good model has low (Test) Loss, which means the Perplexity is $1 = 2^0$
  - A bad model has a high (Test) Loss, which means the Perplexity is $\infty = 2^{\infty}$

# Perplexity For the Uniform Trigram Language Model

- Perplexity is a good way to say one Language Model is better than another
- What happens if $|\{\mathcal{V} \cup \{STOP\}\}| = |\mathcal{V}| + 1$ and we set $\theta(w|v,u) = \frac{1}{|\mathcal{V}|+1}$?
  - Assume we have sentences $\{x^{(i)}\}_{i=1}^{N}$ with length $T_i$
  - We have $\log p(x^{(i)}) = \log \prod_{t=1}^{T_i} \theta(x_t^{(i)}|x_{t-1}^{(i)}, x_{t-2}^{(i)})$
  - Define $T = \sum_{i=1}^{N} T_i$
  - Define Perplexity $= 2^{\frac{N}{T}\mathcal{L}}$ where $\mathcal{L} = -\frac{\sum_{i=1}^{N} \log p(x^{(i)})}{N}$ is the loss
  - Each term $\log p(x^{(i)}) = T_i \log(\frac{1}{|\mathcal{V}|+1})$
  - Then $\frac{N}{T}\mathcal{L} = -\frac{N}{T}\frac{\sum_{i=1}^{N} T_i}{N} \log(\frac{1}{|\mathcal{V}|+1}) = \log(|\mathcal{V}|+1)$
  - So, we have Perplexity is $2^{\log(|\mathcal{V}|+1)} = O(|\mathcal{V}|)$ for the uniform Trigram Model
  - The perplexity can be through of as the effective vocabulary size - We do very poorly with a uniform model, we have not compressed the vocabulary in any way ...

# Neural Language Models in 2003

- What about Neural Network Language Models? We saw one with the gated CNN application
- An MLP Language Model? At a high level the way such a model might work is
  - Fix a convex window $n$
  - Get contiguous words from text such that $y = (w_t)$ and $x = (w_{t-1}, \ldots, w_{t-n+1})$
  - For each word $w \in \mathcal{V}$, assign a unique word embedding vector $C(w)$ of dimension $m$
  - Let $h$ be the number of hidden units, $m$ the word embedding dimension
  - Concatenate the word embeddings for $x$ and set $x \in \mathbb{R}^{(n-1)m}$
  - Set the parameters $b \in \mathbb{R}^{|\mathcal{V}|}$, $W \in \mathbb{R}^{|\mathcal{V}| \times (n-1)m}$, $U \in \mathbb{R}^{|\mathcal{V}| \times h}$, $d \in \mathbb{R}^h$, $H \in \mathbb{R}^{h \times (n-1)m}$ and $C \in \mathbb{R}^{|\mathcal{V}| \times m}$

# Neural Language Models in 2003

- The MLP Language Model is easy to explain if you think of $x$ as the words before the target word $y$
  - Set the parameters $b \in \mathbb{R}^{|\mathcal{V}|}$, $W \in \mathbb{R}^{|\mathcal{V}| \times (n-1)m}$, $U \in \mathbb{R}^{|\mathcal{V}| \times h}$, $d \in \mathbb{R}^h$, $H \in \mathbb{R}^{h \times (n-1)m}$ and $C \in \mathbb{R}^{|\mathcal{V}| \times m}$
  - Set
$$y = b + Wx + U * Tanh(d + Hx) \in \mathbb{R}^{|\mathcal{V}|}$$
  - Return $softmax(y)$ and as usual compare this probability vector against the ground truth : "What word goes after $(w_{t-1}, \ldots, w_{t-n+1})$?"
- Notice: the above has skip connections: $x \to x + f(x)$ so that this allows easier gradient flow
- Also notice, the model got the best Perplexity at the time, beating several other popular benchmarks

# Neural Language Model: Visuals and Results

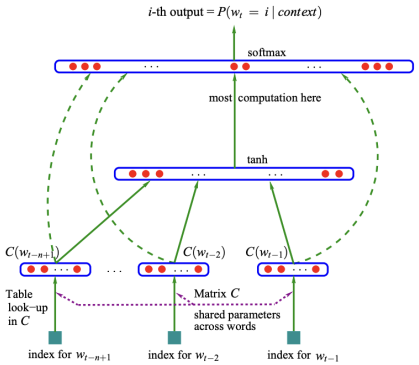



Figure 1: Neural architecture: $f(i, w_{t-1}, \cdots, w_{t-n+1}) = g(i, C(w_{t-1}), \cdots, C(w_{t-n+1}))$ where $g$ is the neural network and $C(i)$ is the $i$-th word feature vector.

| | n | c | h | m | direct | mix | train. | valid. | test. |
|---|---|---|---|---|---|---|---|---|---|
| MLP1 | 5 | | 50 | 60 | yes | no | 182 | 284 | 268 |
| MLP2 | 5 | | 50 | 60 | yes | yes | | 275 | 257 |
| MLP3 | 5 | | 0 | 60 | yes | no | 201 | 327 | 310 |
| MLP4 | 5 | | 0 | 60 | yes | yes | | 286 | 272 |
| MLP5 | 5 | | 50 | 30 | yes | no | 209 | 296 | 279 |
| MLP6 | 5 | | 50 | 30 | yes | yes | | 273 | 259 |
| MLP7 | 3 | | 50 | 30 | yes | no | 210 | 309 | 293 |
| MLP8 | 3 | | 50 | 30 | yes | yes | | 284 | 270 |
| MLP9 | 5 | | 100 | 30 | no | no | 175 | 280 | 276 |
| MLP10 | 5 | | 100 | 30 | no | yes | | 265 | **252** |
| Del. Int. | 3 | | | | | | 31 | 352 | 336 |
| Kneser-Ney back-off | 3 | | | | | | | 334 | 323 |
| Kneser-Ney back-off | 4 | | | | | | | 332 | 321 |
| Kneser-Ney back-off | 5 | | | | | | | 332 | 321 |
| class-based back-off | 3 | 150 | | | | | | 348 | 334 |
| class-based back-off | 3 | 200 | | | | | | 354 | 340 |
| class-based back-off | 3 | 500 | | | | | | 326 | **312** |
| class-based back-off | 3 | 1000 | | | | | | 335 | 319 |
| class-based back-off | 3 | 2000 | | | | | | 343 | 326 |
| class-based back-off | 4 | 500 | | | | | | 327 | 312 |
| class-based back-off | 5 | 500 | | | | | | 327 | 312 |

Table 1: Comparative results on the Brown corpus. The deleted interpolation trigram has a test perplexity that is 33% above that of the neural network with the lowest validation perplexity. The difference is 24% in the case of the best n-gram (a class-based model with 500 word classes). $n$ : order of the model. $c$ : number of word classes in class-based n-grams. $h$ : number of hidden units. $m$ : number of word features for MLPs, number of classes for class-based n-grams. *direct*: whether there are direct connections from word features to outputs. *mix*: whether the output probabilities of the neural network are mixed with the output of the trigram (with a weight of 0.5 on each). The last three columns give perplexity on the training, validation and test sets.

# Language Modeling

- But, all these basic models had a problem, however: limited (fixed) context
- Can we build models with the potential of having unlimited context?
- We have via Bayes' rule that

$$p(y_1, \ldots, y_T) = p(y_1)p(y_2|y_1)\ldots p(y_T|y_{T-1}, \ldots, y_1)$$

- At a high level, any of the CNN/ MLP / N-Gram based models approximates the above, they assume for example that

$$p(y_t|y_{t-1}, \ldots, y_1) \sim p(y_t|y_{t-1}, \ldots, y_{t-k})$$

- The models we look at today do not need such approximations ...

# Outline

- Language Modeling
- Recurrent Neural Networks
- LSTM
- Decoding

# Recurrent Neural Networks: example data

- One way get training data for language modeling is to grab chunks of contiguous text $x = (x_0, x_2, \ldots, x_{T-1})$ and try to predict the shifted version of this text $y = (x_1, x_2, \ldots, x_T)$
- For example, you want to map "START The man walks" to "The man walks END"
- Here $x = (START, "The", "man", "walks")$ and $y = ("The", "man", "walks", END)$
- We'd like to have a model that feeds in each $x_t$ in sequence and using this $x_t$ (and past history) predicts $y_t$
- Some notes:
  - In the example above, the Loss should have 4 terms
  - Feeding data into this model should be sequential, from left to right
  - We need a way to pass in new data and carry forward older data
  - This is Self-Supervised Learning: we took some raw data and built $x$ and $y$ from it out of thin air

# Recurrent Neural Networks

- In summary we want a model so that we can build

$$p(y_1, y_2, \ldots, y_T)$$

  with minimal approximations
- Using Bayes' rule, this is equal to

$$p(y_1)p(y_2|y_1)\ldots p(y_T|y_{T-1}, \ldots, y_1)$$

- We'd like to NOT approximate this, we'd like to have in some sense a very large or an infinite context
- We want models that can truly express terms like

$$p(y_T|y_{T-1}, \ldots, y_1)$$

- RNN's promise this, but they have some issues as we'll see

# Recurrent Neural Networks: Recursions

- Each time step of an RNN related $h_t$ to $h_{t-1}$ and $x_t$
  - At each time step, we pass in $x_t$ (the current word)
  - Additionally, also use the previous $h_{t-1}$ (a past history)
- We have

$$h_t = tanh(W_h h_{t-1} + W_x x_t + b_h)$$

- At each time step, we have $\hat{y}_t = F(W_y h_t + b_y)$ where $F$ is usually a sigmoid or softmax (over $|\mathcal{V}|$ possible words)
- We initialize $h_0$ appropriately
- At each time step, we have a Loss $\mathcal{L}_t(\hat{y}_t, y_t)$ which is usually a Cross-Entropy
- The full loss per training pair

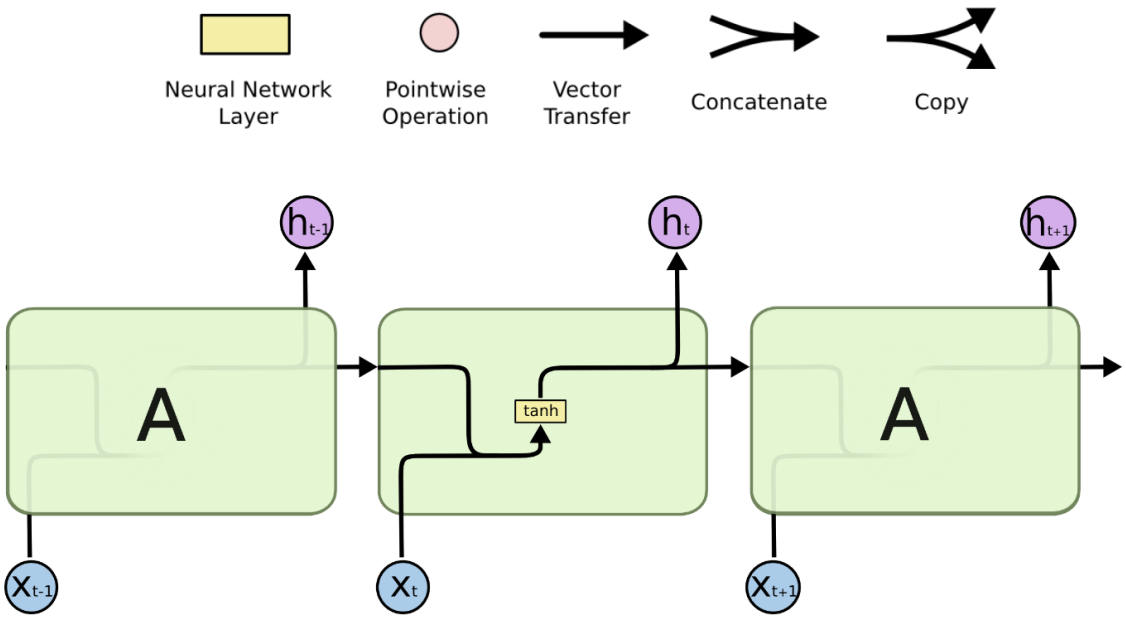
$$((x_1, x_2, \ldots, x_T), (y_1 = x_2, y_2 = x_3, \ldots, y_T = x_{T+1}))$$

pair is an average of the loss across all time steps

$$\frac{\sum_{t=1}^{T} \mathcal{L}_t(\hat{y}_t, y_t)}{T}$$

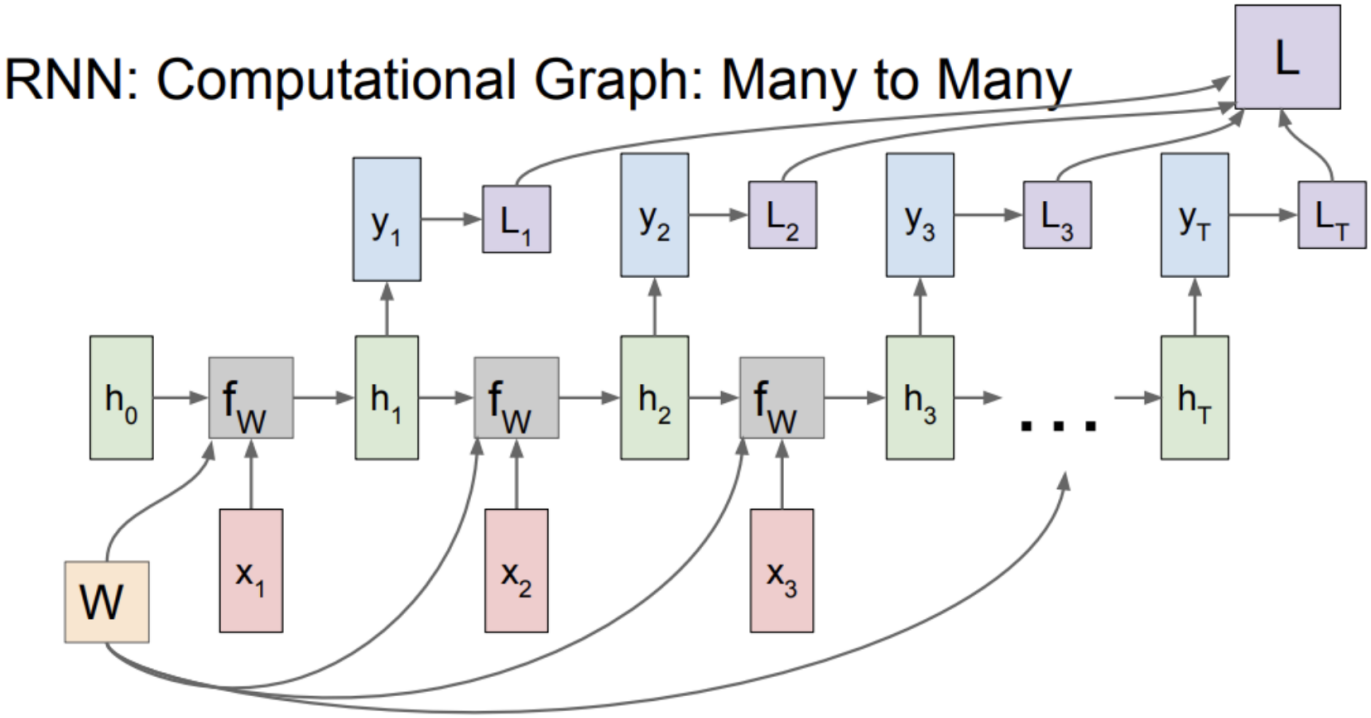
# Recurrent Neural Networks: Recursions

- A picture of a RNN might look like this:



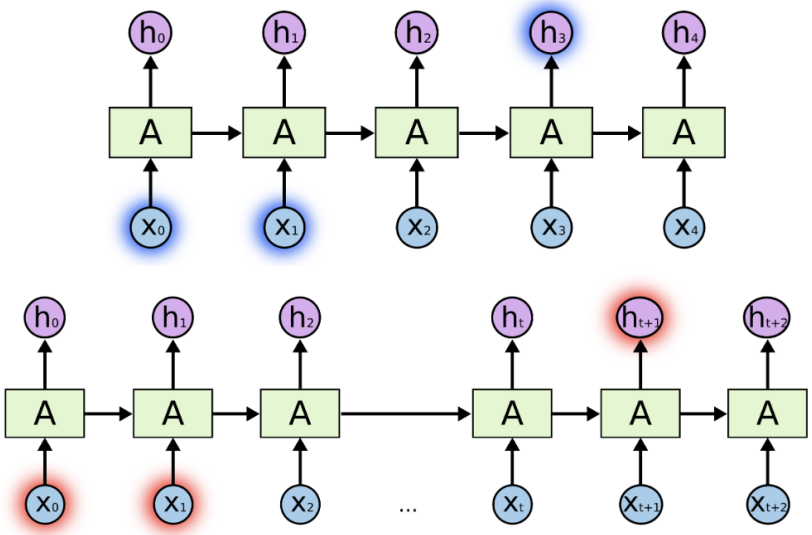

The repeating module in a standard RNN contains a single layer.

# Recurrent Neural Networks: Computational Graph



# Recurrent Neural Networks: Recursions

- As the data gets longer, an RNN can no longer propagate dependencies between things far in the future and elements earlier on




Why?

# Recurrent Neural Networks: Recursions

- Let's do a small example where $W_h = a$, $W_x = b$ and $b_h = c$ are all scalars and we have the recursion $h_t = ah_{t-1} + bx_t + c$
- Assume also we have $\hat{y}_t = dh_t$ and the loss is $\mathcal{L}_t(\hat{y}_t, y_t) = (\hat{y}_t - y_t)^2$
- Here, $a, b, c, d$ are parameters of dimension 1
- What is $\frac{\partial \mathcal{L}_1}{\partial a}$?
- Here we have $\frac{\partial \mathcal{L}_1}{\partial a} = \frac{\partial \mathcal{L}_1}{\partial \hat{y}_1} \frac{\partial \hat{y}_1}{\partial h_1} \frac{\partial h_1}{\partial a} = 2(\hat{y}_1 - y_1)dh_0$

# Recurrent Neural Networks: Recursions

- Let's do a small example where $W_h = a$, $W_x = b$ and $b_h = c$ are all scalars and we have the recursion $h_t = ah_{t-1} + bx_t + c$
- Assume also we have $\hat{y}_t = dh_t$ and the loss is $\mathcal{L}_t(\hat{y}_t, y_t) = (\hat{y}_t - y_t)^2$
- What is $\frac{\partial \mathcal{L}_2}{\partial a}$?
- Here we have $\frac{\partial \mathcal{L}_2}{\partial a} = \frac{\partial \mathcal{L}_2}{\partial \hat{y}_2} \frac{\partial \hat{y}_2}{\partial h_2} \frac{\partial h_2}{\partial a}$
- This is actually $\frac{\partial \mathcal{L}_2}{\partial \hat{y}_2} \frac{\partial \hat{y}_2}{\partial h_2} [\frac{\partial h_2}{\partial a} + \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial a}]$
- This is $2(\hat{y}_2 - y_2)d(h_1 + ah_0)$
- Note the appearance of the $a$ term

# Recurrent Neural Networks: Recursions

- What is $\frac{\partial \mathcal{L}_4}{\partial a}$?
- Here we have $\frac{\partial \mathcal{L}_4}{\partial a} = \frac{\partial \mathcal{L}_4}{\partial \hat{y}_4} \frac{\partial \hat{y}_4}{\partial h_4} \frac{\partial h_4}{\partial a}$
- This is actually

$$\frac{\partial \mathcal{L}_4}{\partial a} = \sum_{t=1}^{3} \frac{\partial \mathcal{L}_4}{\partial \hat{y}_4} \frac{\partial \hat{y}_4}{\partial h_4} \frac{\partial h_4}{\partial h_t} \frac{\partial h_t}{\partial a} + \frac{\partial \mathcal{L}_4}{\partial \hat{y}_4} \frac{\partial \hat{y}_4}{\partial h_4} \frac{\partial h_4}{\partial a}$$

- Simplified, this is

$$2(\hat{y}_4 - y_4) da^3 h_0 + 2(\hat{y}_4 - y_4) da^2 h_1 + 2(\hat{y}_4 - y_4) da h_2 + 2(\hat{y}_4 - y_4) dh_3$$

- What happens when $a$ is small or large?

# Recurrent Neural Networks: Gradients Issues

- We have the gradient $\frac{\partial \mathcal{L}_4}{\partial a}$ was

$$2(\hat{y}_4 - y_4)da^3 h_0 + 2(\hat{y}_4 - y_4)da^2 h_1 + 2(\hat{y}_4 - y_4)dah_2 + 2(\hat{y}_4 - y_4)dh_3$$

- If $a$ is small, we have that the term measuring the dependency between the current time step and the one in the past is less and less
- This is called the vanishing gradient problem
- If $a$ is too large, we'll have an infinite gradient, another problem
- This is called the exploding gradient problem

# Recurrent Neural Networks: Gradients Issues

- You can also get the recursion over and over and see what happens
- You can also see this (do the recursion over and over):

$$h_5 = bx_5 + bax_4 + ba^2x_3 + ba^3x_2 + \dots$$

- If $h_t$ is going to predict $y_t$, we clearly see old information "vanishes" if $a$ is small
- In some sense this illustrates the problem

# Recurrent Neural Networks: Solving Gradient Issues

- The exploding gradient problem can be solved by gradient clipping
- The vanishing gradient problem might be solved by other ways
  - Residual connections
  - Xavier Initialization (some sort of smart initialization)
  - Momentum based optimizers (Adam)
- None of the above are principled, they "might" work
- In practice, vanilla RNNs do not perform as good as they theoretically can due to the vanishing gradient problem
- More complicated (LSTM, Transformers, Attention) models are used

# Recurrent Neural Networks: Truncated BP

- Doing back propagation in RNN models can be expensive since you need to go all the way back to $t = 0$
- We generally have

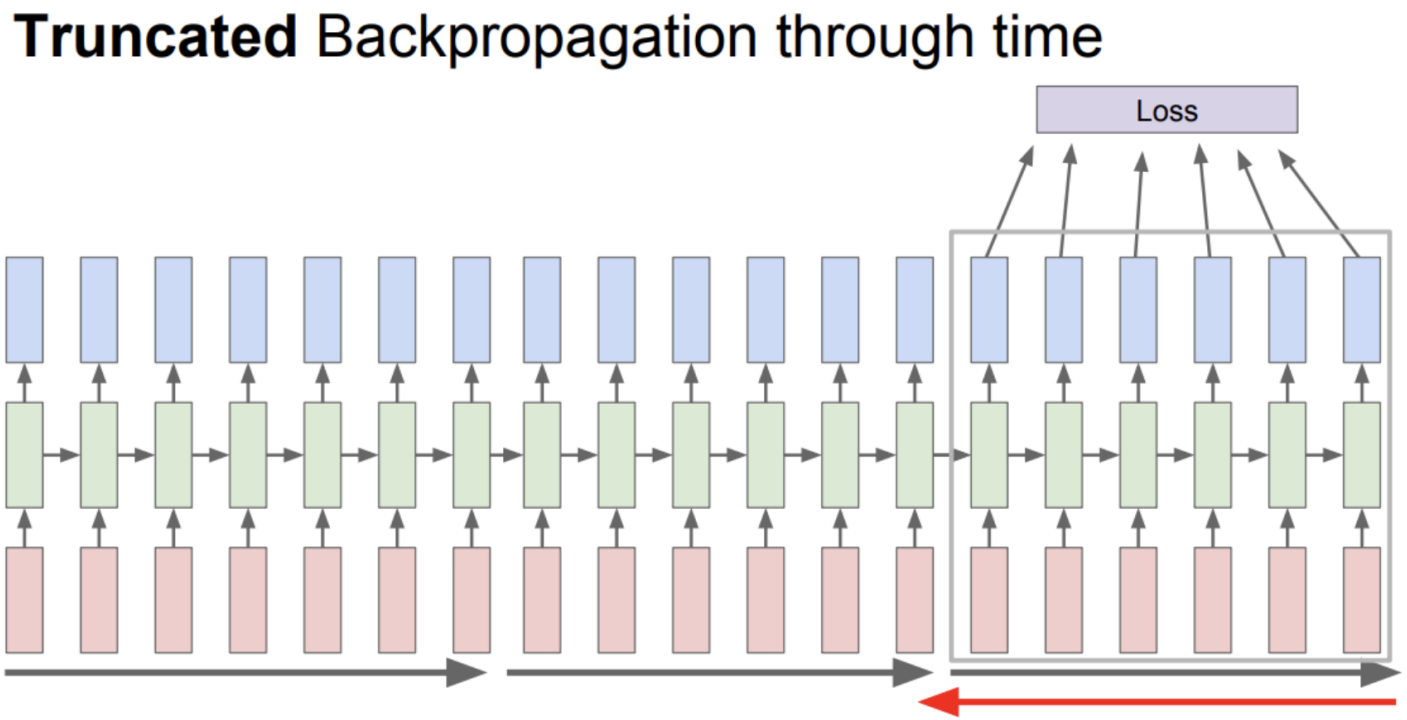
$$\frac{\partial \mathcal{L}_t}{\partial a} = \sum_{s=1}^{t} \frac{\partial \mathcal{L}_t}{\partial \hat{y}_t} \frac{\partial \hat{y}_t}{\partial h_t} \frac{\partial h_t}{\partial h_s} \frac{\partial h_s}{\partial a}$$

  - I use $\frac{\partial h_t}{\partial h_t} = 1$ for the case $s = t$
- One idea is to simplify the above to be

$$\frac{\partial \mathcal{L}_t}{\partial a} = \sum_{s=t-k}^{t} \frac{\partial \mathcal{L}_t}{\partial \hat{y}_t} \frac{\partial \hat{y}_t}{\partial h_t} \frac{\partial h_t}{\partial h_s} \frac{\partial h_s}{\partial a}$$

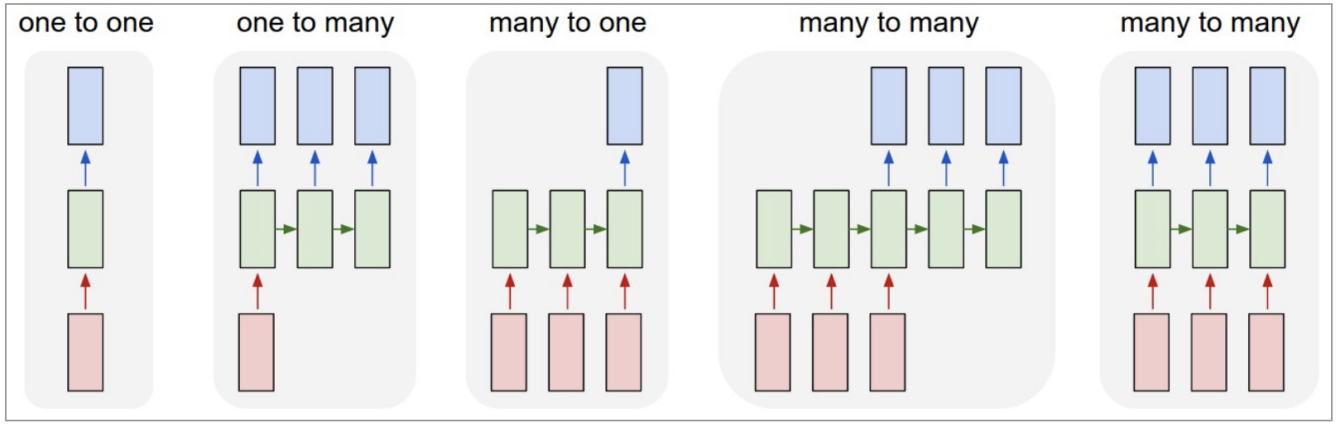
- Thus, for each term in the loss $\mathcal{L}_t$, we only look back a fixed window when computing the gradients
- This works somewhat, but now we are back to the finite context case

# Recurrent Neural Networks: Truncated BP Visual



# Recurrent Neural Networks: Variants

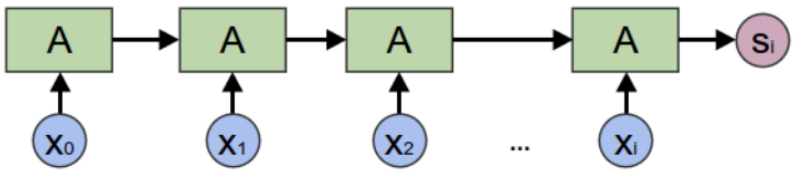
- Many data and modeling situations can be addressed via Recurrent Neural Networks

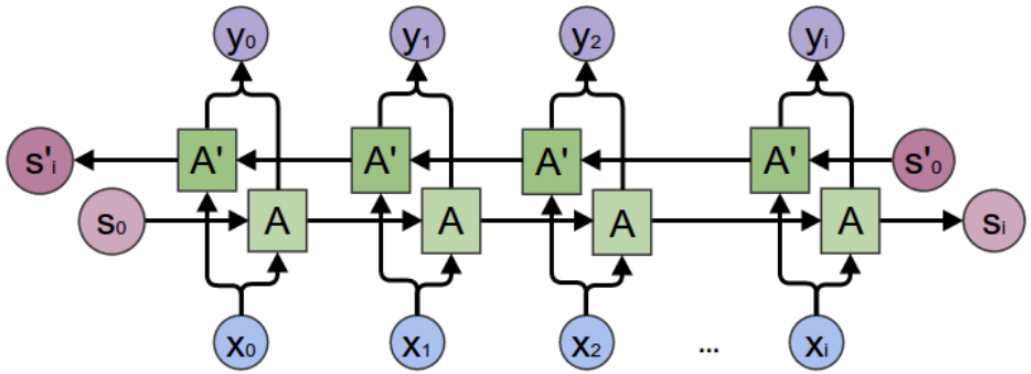



Each rectangle is a vector and arrows represent functions (e.g. matrix multiply). Input vectors are in red, output vectors are in blue and green vectors hold the RNN's state (more on this soon). From left to right: **(1)** Vanilla mode of processing without RNN, from fixed-sized input to fixed-sized output (e.g. image classification). **(2)** Sequence output (e.g. image captioning takes an image and outputs a sentence of words). **(3)** Sequence input (e.g. sentiment analysis where a given sentence is classified as expressing positive or negative sentiment). **(4)** Sequence input and sequence output (e.g. Machine Translation: an RNN reads a sentence in English and then outputs a sentence in French). **(5)** Synced sequence input and output (e.g. video classification where we wish to label each frame of the video). Notice that in every case are no pre-specified constraints on the lengths sequences because the recurrent transformation (green) is fixed and can be applied as many times as we like.

# Recurrent Neural Networks: Bidirectional RNN

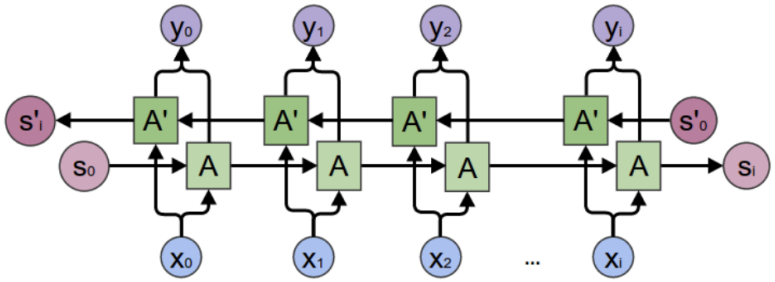
- You can also have models which have context on both sides
- Imagine the sentiment analysis of "The great movie was actually really just expensive and bad."
- You'd like to know that "great" has a bit of irony in it and it contrasts with "expensive and bad"
- Sentiment analysis is one example where a bidirectional RNN might be a good idea: each word has the best context of those in the sentence!



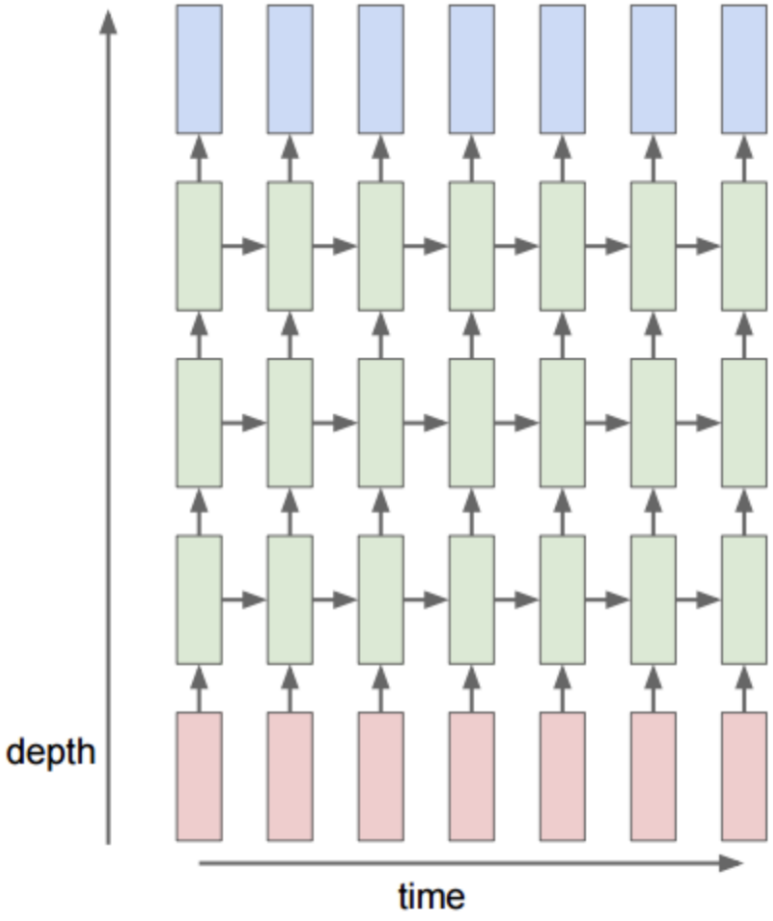
# Recurrent Neural Networks: Bidirectional RNN

- When you are generating text, you can't use a truly bidirectional RNN (given our architecture) since then you are cheating
- For that, you'd need to run a model left-to-right and another (independent) model right-to-left and then fuse the results to get (weak) bidirectionally
- Imagine "He painted the wall with a brush, not a spray can."
  - $x = (START, he, painted, \ldots, a, spray, can)$
  - $y = (he, painted, \ldots, not, a, spray, can, END)$
  - A left-to-right RNN might initially struggle to determine what "painted the" refers to until it reaches "wall".
  - A bidirectional RNN, however, can look ahead and see "with a brush" while also considering "painted the", making it easier to correctly infer that "wall" is the object being painted, rather than, say, "a picture" or "a model".



# Recurrent Neural Networks: Multilayer RNN

- Multilayer RNN is also possible, but now you have many more parameters
- Note that
  - more layers or depth = vertical
  - more time = horizontal



# Outline

- Language Modeling
- Recurrent Neural Networks
- LSTM
- Decoding

# LSTM

- RNN suffer from vanishing gradient problems
- Because of this, people have tried to come up with variants that resolve this issue
- LSTM (1997 !) are one of the most popular variants
- Empirically, an LSTM has been shown to have a memory of about 100 time steps, whereas a RNN has about just 7

# LSTM

- Dynamics of the LSTM are as follows
- We still have data $x = (x_0, x_1, \ldots, x_T)$ but now we have a more complicated interaction
- Specifically, each time step not only has $h_t$, but now there is a cell state $c_t$ that is carried forward
- We have

$$(f_t, i_t, o_t, g_t) = W_h h_{t-1} + W_x x_t + b$$

- Where the state is

$$c_t = \sigma(f_t) \odot c_{t-1} + \sigma(i_t) \odot tanh(g_t)$$

- And the hidden state is

$$h_t = \sigma(o_t) \odot tanh(c_t)$$

# LSTM

- The key new feature here is

$$c_t = \sigma(f_t) \odot c_{t-1} + \sigma(i_t) \odot tanh(g_t)$$

  - $\sigma(f_t)$ is the forget gate
  - $\sigma(i_t)$ is the update gate
- This cell state is a combination of old information and new information: the sole purpose of this state is to help the network not forget!
- And the hidden state is $h_t = \sigma(o_t) \odot tanh(c_t)$ is a function of new information and the new/old information in the cell state
- As we will show, this affects the gradients: there is less chance of vanishing gradient phenomenon and a current loss update can use information from far out in the past!

# LSTM: Parameters

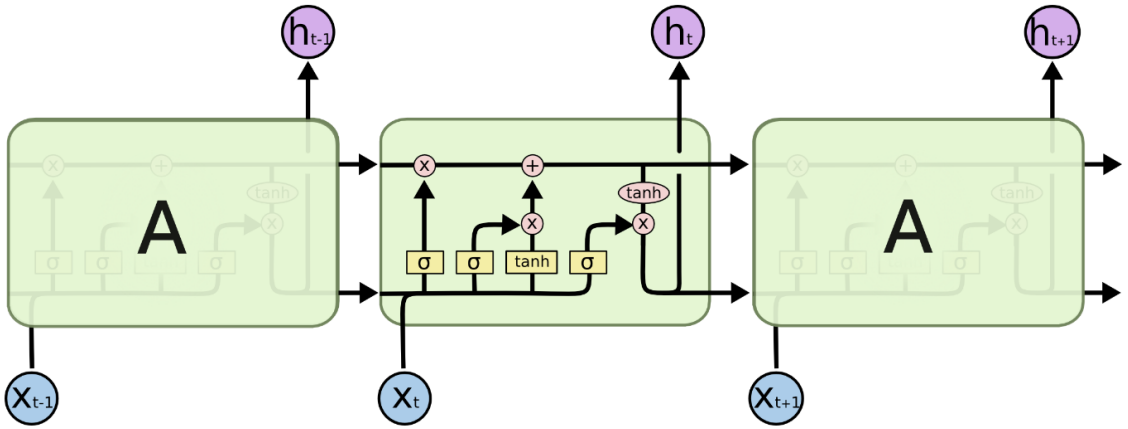
- In the above
- $W_h \in \mathbb{R}^{d_h \times 4d_h}$
- $W_x \in \mathbb{R}^{d_x \times 4d_h}$
- $b \in \mathbb{R}^{4d_h}$
- $h_0, c_0 \in \mathbb{R}^{d_h}$
- $\sigma$ is the sigmoid
- $\odot$ is the Hadamard product, $[A \odot B]_{ij} = A_{ij} B_{ij}$

# LSTM: Parameters

- So we can be more specific with the parameters
- Note that in reality we can think of $W_x = [W_{fx}, W_{ix}, W_{ox}, W_{gx}]$
- We can thus break up the weight matrix into blocks, with a specific block being for a specific parameter
- A similar idea applied to $W_h$ and $b = [b_f, b_i, b_o, b_g]$

# LSTM: Pictures

- Here is a nice picture of what is going on in an LSTM




The repeating module in an LSTM contains four interacting layers.

# LSTM: Idea

- The big idea of LSTM networks is that the added $c_t$ allows the networks to remember things
- Put another way, if we write out gradients, there is a greater chance of long term dependencies
- Why?
- Recall, terms of the type $\frac{\partial h_t}{\partial h_{t-k}}$ are critical to gradient estimates for the RNN, let's focus on them since they are a (critical) part of all gradients
- For an LSTM, we also have $\frac{\partial c_t}{\partial c_{t-k}}$ to consider

# RNN: Gradients

- Consider again an RNN and simplify it a bit so that $h_t = \sigma(W_h h_{t-1})$
- Recall $\frac{d\sigma(z)}{dz} = \sigma(z)(1 - \sigma(z))$
- If we do a gradient computation we get that

$$\frac{\partial h_t}{\partial h_{t-k}} = \frac{\partial h_t}{\partial h_{t-1}} \cdots \frac{\partial h_{t-k+1}}{\partial h_{t-k}} = \prod_{s=t-k}^{t-1} W_h \sigma(W_h h_s)(1 - \sigma(W_h h_s))$$

- This is

$$\frac{\partial h_t}{\partial h_{t-k}} = W_h^k \prod_{s=t-k}^{t-1} \sigma(W_h h_s)(1 - \sigma(W_h h_s))$$

# RNN: $h_t$ Gradients

- This is

$$\frac{\partial h_t}{\partial h_{t-k}} = W_h^k \prod_{s=t-k}^{t-1} \sigma(W_h h_s)(1 - \sigma(W_h h_s))$$

- When $W_h h_s$ is large or small, we get vanishing gradients
- Even when $W_h h_s = 0$, the maximum point of $\sigma(x)(1 - \sigma(x))$, we get $1/4$, so we get a small product
- The term $W_h^k$ can also cause vanishing or exploding gradients, mainly due to bad initialization
- Upshot: A lot of things can go wrong, and usually do

# LSTM: $c_t$ Gradients

- Now look at the LSTM cell
  $c_t = \sigma(f_t) \odot c_{t-1} + \sigma(i_t) \odot tanh(g_t)$
- What is $\frac{\partial c_t}{\partial c_{t-k}}$ ?
- Ignoring terms that have to do with $x$ and $b_f$, this gives

$$\frac{\partial c_t}{\partial c_{t-k}} = \frac{\partial c_t}{\partial c_{t-1}} \cdots \frac{\partial c_{t-k+1}}{\partial c_{t-k}} = \prod_{s=t-k}^{t-1} \sigma(f_s) = \prod_{s=t-k}^{t-1} \sigma(W_{fh} h_s + \ldots)$$

- In this case, gradients only vanish if $W_{fh} h_s$ is very small: we have much less things that can go wrong!
  - Also, since the $\sigma$ function squashes, the gradient will not explode because $W_{fh} h_s$ is very large we'll get 1
  - We don't have a bad "$W_h^k$" term like we had before

# LSTM: $h_t$ Gradients

- Now look at the LSTM hidden unit $h_t = \sigma(o_t) \odot tanh(c_t)$
- Keep in mind the LSTM cell
  $c_t = \sigma(f_t) \odot c_{t-1} + \sigma(i_t) \odot tanh(g_t)$
- Recall $\frac{d\sigma(z)}{dz} = \sigma(z)(1 - \sigma(z))$ and $\frac{dtanh(z)}{dz} = 1 - tanh^2(z)$
- What about the term $\frac{\partial h_t}{\partial h_{t-k}} = \frac{\partial h_t}{\partial h_{t-1}} \dots \frac{\partial h_{t-k+1}}{\partial h_{t-k}}$ ?

# LSTM: $h_t$ Gradients

- If we focus just on $\frac{\partial h_t}{\partial h_{t-1}}$ we have to simplify

$$\partial(\sigma(o_t) \odot tanh(c_t))/\partial h_{t-1}$$

- This is

$$\frac{\partial \sigma(o_t)}{\partial h_{t-1}} tanh(c_t) + \sigma(o_t) \frac{\partial tanh(c_t)}{\partial h_{t-1}}$$

- Which is

$$\sigma(o_t)(1 - \sigma(o_t)) tanh(c_t) W_{oh} +$$
$$\sigma(o_t)(1 - tanh^2(c_t))[\frac{\partial \sigma(f_t)}{\partial h_{t-1}} c_{t-1} + \frac{\partial(\sigma(i_t) \odot tanh(g_t))}{\partial h_{t-1}}]$$

# LSTM: $h_t$ Gradients

- We want to simplify the term

$$\frac{\partial \sigma(f_t)}{\partial h_{t-1}} c_{t-1} + \frac{\partial(\sigma(i_t) \odot tanh(g_t))}{\partial h_{t-1}}$$

- The left term simplifies to

$$\frac{\partial \sigma(f_t)}{\partial h_{t-1}} c_{t-1} = \sigma(f_t)(1 - \sigma(f_t)) W_{fh} c_{t-1}$$

- Th right term simplifies to

$$\begin{aligned} \frac{\partial(\sigma(i_t) \odot tanh(g_t))}{\partial h_{t-1}} &= \sigma(i_t)(1 - \sigma(i_t)) W_{ih} tanh(g_t) \\ &+ (1 - tanh^2(g_t)) \sigma(i_t) W_{gh} \end{aligned}$$

# LSTM: $h_t$ Gradients

- Putting this together, we get

$$
\begin{aligned}
&tanh(\bullet)\sigma(\bullet)(1-\sigma(\bullet))W_{oh}+ \\
&\qquad \sigma(\bullet)(1-tanh^2(\bullet))[c_{t-1}\sigma(\bullet)(1-\sigma(\bullet))W_{fh}+ \\
&\qquad \sigma(\bullet)(1-tanh^2(\bullet))W_{gh}+tanh(\bullet)\sigma(\bullet)(1-\sigma(\bullet))W_{ih}]
\end{aligned}
$$

- Let $\gamma(\bullet)$ be any generic term like $\sigma$ or $tanh$, which are all bounded and assume $\gamma^2(\bullet)=\gamma(\bullet)$
- We get

$$\gamma(\bullet)W_{oh}+\gamma(\bullet)[c_{t-1}\gamma(\bullet)W_{fh}+\gamma(\bullet)W_{gh}+\gamma(\bullet)W_{ih}]$$

# LSTM: $h_t$ Gradients

- Using $\gamma^2 = \gamma$, this is further reduced to

$$\gamma(\bullet)W_{oh} + c_{t-1}\gamma(\bullet)W_{fh} + \gamma(\bullet)W_{gh} + \gamma(\bullet)W_{ih}$$

- So we have

$$\frac{\partial h_t}{\partial h_{t-k}} = \prod_{s=t-k}^{t-1} [\gamma(\bullet)W_{oh} + c_{t-1}\gamma(\bullet)W_{fh} + \gamma(\bullet)W_{gh} + \gamma(\bullet)W_{ih}]$$

- So, although $\frac{\partial c_t}{\partial c_{t-k}}$ has no exponential products, $\frac{\partial h_t}{\partial h_{t-k}}$ does, we can still have vanishing gradients for this term
- We must still be careful with initialization, and we might need to do gradient clipping
- However, $h_t$ being unsafe from vanishing gradients is not as critical here as it is for a standard RNN; $c_t$ can be used to pass on long range dependencies - the model has this capacity and $h_t$ can use that

# LSTM: Peephole Connected LSTM

- Generalization of LSTM: Peephole Connections
- We now pass the old state $c_{t-1}$ in:

$$(f_t, i_t) = W_h h_{t-1} + W_x x_t + W_c c_{t-1} + b$$

- $g_t = W_{gh} h_{t-1} + W_{gx} x_t + b_g$ (same as before)
- $c_t = \sigma(f_t) \odot c_{t-1} + \sigma(i_t) \odot tanh(g_t)$ (same as before)
- We now have $c_t$ is used for the output:
  $o_t = W_{oh} h_{t-1} + W_{ox} x_t + W_{oc} c_t + b_o$
- The idea of these models is to fix the case when $\sigma(o_t) \sim 0$ because then $h_t = \sigma(o_t) \odot tanh(c_t) \sim 0$
- Allowing the gates to have access to the cell state has been shown empirically to fix this issue

# GRU: Simpler LSTM

- Simplification of LSTM: GRU
- A GRU defines reset and update gate logits via

$$(r_t, z_t) = W_h h_{t-1} + W_x x_t + b$$

- For a GRU, we have $g_t = W_{gh}(\sigma(r_t)h_{t-1}) + W_{gx}x_t + b_g$
- Now, $h_t = \sigma(z_t) \odot tanh(g_t) + (1 - \sigma(z_t)) \odot h_{t-1}$
  - GRU have a single state
  - They combine the gates $\sigma(f_t)$ and $\sigma(i_t)$ into one gate $\sigma(z_t)$
  - The reset gate $\sigma(r_t)$ allows the GRU to use or ignore the previous state when computing the next candidate $g_t$
- GRU models offer less parameters than a LSTM and have been shown to do about as well

# LSTMs now

- Are LSTMs dead now?
- Sort of, but there are some ideas which may bring them back
- Why are they dead?
- Main problem: parallelism
  - If I give you a batch of sentences, to compute this batch you need to go one by one though each word in each sentence to get the hidden states
  - You can parallelize across difference sentences, but not *within* each sentence
  - GPU cannot be used as well as possible
- Note: People have been trying, see references

# Outline

- Language Modeling
- Recurrent Neural Networks
- LSTM
- Decoding

# Decoding: Example

- Once we train a model, we have the problem of decoding: How do we generate text using the model?
- Usually, we start with a context like "The man walks" or a special start token "*START*" and begin decoding with the model
- One very simple variant: greedy decoding
- This is one of the oldest and easiest heuristics

# Decoding: Greedy Search Example

- Basic idea: Always pick the highest probability next word and use that as your next word
- Once picked, add the word to your context and repeat until some terminal condition happens
- Big idea is that, for example, we have

$$p(The, man, walks, y) = p(y|The, man, walks)p(The, man, walks)$$

- We can just focus on

$$p(y|The, man, walks)$$

if we want to maximize

$$p(The, man, walks, y)$$

# Decoding: Greedy Search Example

- So suppose the context is "*The man walks*"
- Get

$$argmax_y p(y|The, man, walks)$$

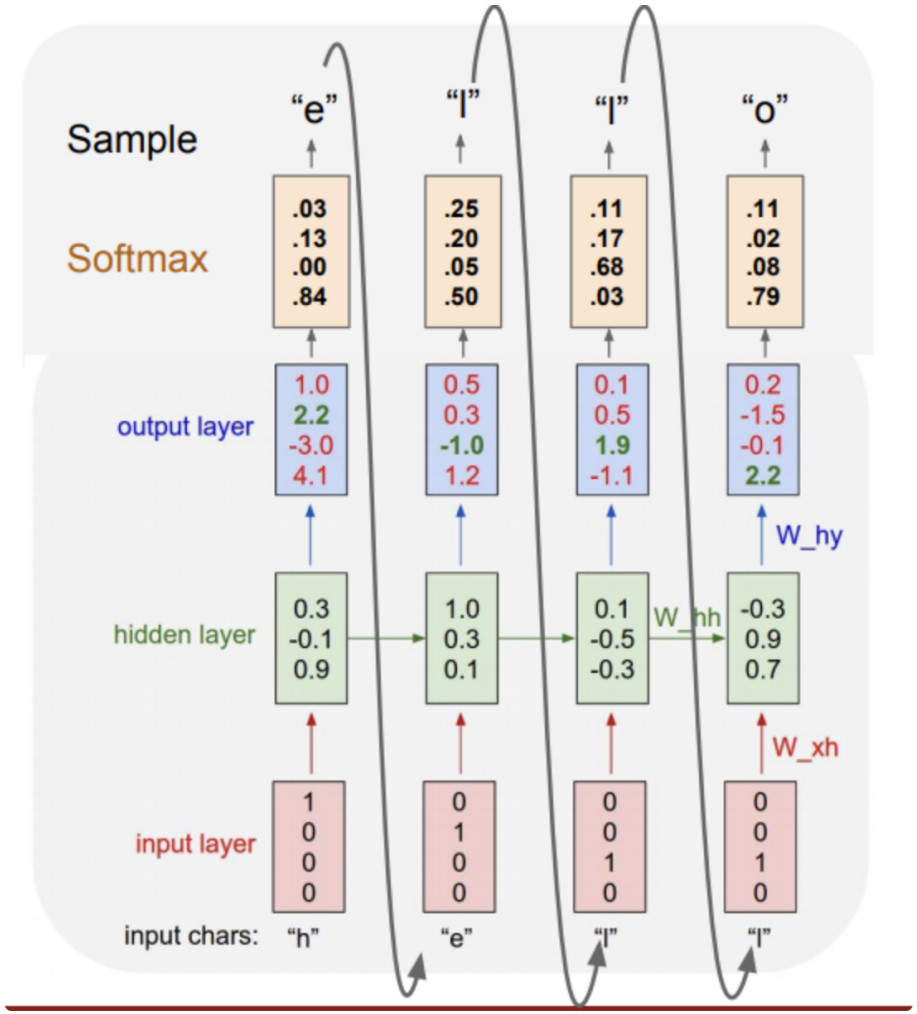
  and suppose this is "*down*"
  - Note that technically we don't work with strings, but the integers that each string represent
- Now the context is "The man walks down"
- Get

$$argmax_y p(y|The, man, walks, down)$$

  and suppose this is "*the*"
- Now the context is "*the*"
- Etc. Repeat this until the context is done or until a special end "*END*" token is generated
- Once picked, add the word to your context and repeat over and over

# Decoding: Random Decoding Example

Greedy is no the only way, we can also pick a random next word given past words




But, random can be bad also; we can improve this ...

# Decoding: Greedy is not Optimal, Random might be too "random"

- Remember we really want to solve

$$\max_{y_1, y_2, \ldots, y_T} (p(y_1, y_2, \ldots, y_T))$$

- Greedy, generally, will not give the optimal solution - we keep just one choice!
- Also, human generated text is not generated this way: you don't always pick the word with the next highest probability to generate the things you say
- Sometimes, you make a lower probability choice but in the long run this leads to higher probability choices overall
- Think of generating this word by word: "The Himalayan Mountains are very nice"
- Another idea: Beam Search

# Decoding: Beam Search

- Instead of picking the next highest probability word to add to your context, store a beam of size $k$ as your set of candidates
- Example: Assume $k = 2$ and your context is again "*The man walks*"
- When we look at

$$argmax_y p(y|The, man, walks)p(The, man, walks)$$

  we don't just pick $y =$ "*down*", we also pick "*fast*", which has the second highest probability
- Now you have two context, "*The man walks down*" and "*The man walks fast*"
- Next time, consider

$$p(y|The, man, walks, down)p(The, man, walks, down)$$

  and

$$p(y|The, man, walks, fast)p(The, man, walks, fast)$$

  and get the $k = 2$ maximum probability tokens

# Decoding: Beam Search

- Note that now we have to consider $k * |\mathcal{V}|$ scores and we want to just select $k = 2$
- Note that we might have that among

$$p(y|The, man, walks, down)p(The, man, walks, down)$$

and

$$p(y|The, man, walks, fast)p(The, man, walks, fast)$$

the first case has the highest probability next words

- I.e. We might have that

$$p(The, man, walks, down, the)$$

and

$$p(The, man, walks, down, into)$$

are the highest probabilities

# Decoding: Beam Search

- Effectively, the new context becomes

$$\text{"}\textit{The man walks down the}\text{"}$$

and

$$\text{"}\textit{The man walks down into}\text{"}$$

so that

$$\text{"}\textit{The man walks fast}\text{"}$$

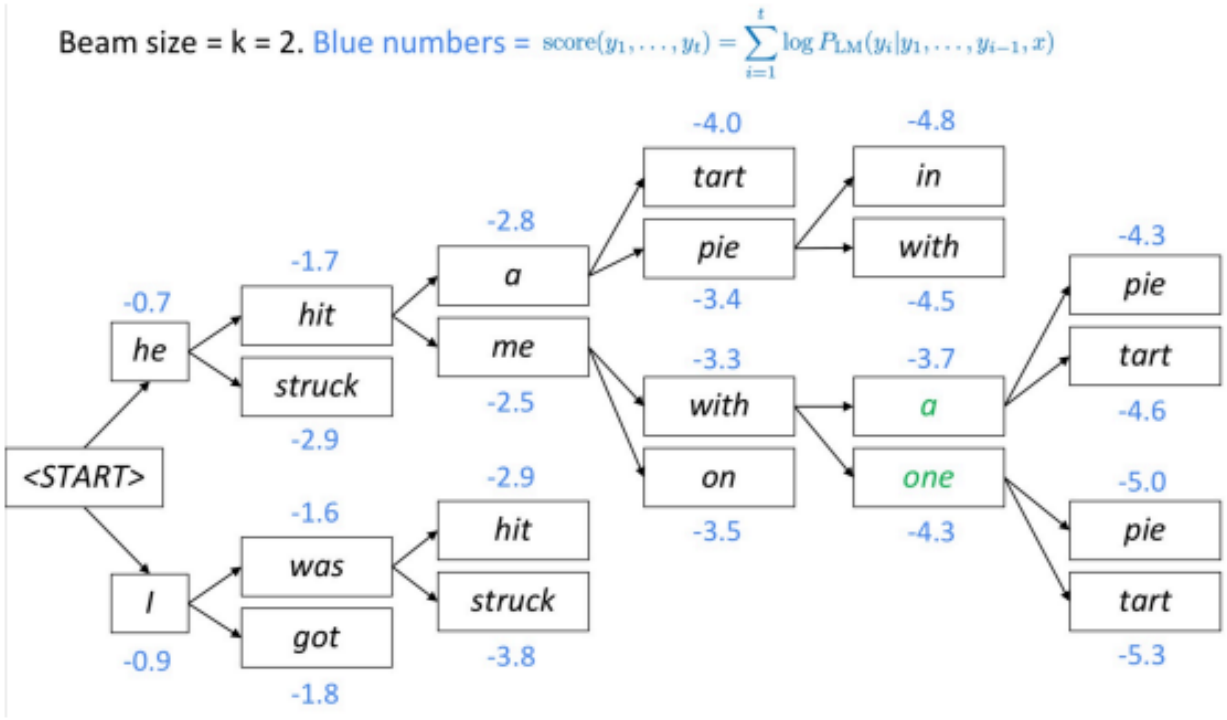
is dropped

- Usually, Beam Search uses the trick below since it avoids the product of small numbers

$$\log p(y_1, y_2, \ldots, y_T) = \sum_{t=1}^{T} \log p(y_t | y_{t-1}, \ldots, y_1)$$

# Decoding: Beam Search

- Usually, Beam Search uses the trick below (work with log-probabilities); an example:



# References


- Backpropogating an LSTM: A Numerical Example
- Understanding LSTM Networks
- Character Aware Neural Language Models
- Neural Machine Translation
- Andrej Karpathy's BLOG on RNNs
- Machine Learning Notebook
- Where RNN all we need? Trying to get Transformer performance w RNN